

Information Processing 1 — Week 4 exercises

Complete each exercise on your own laptop before the end of the exercise class. When you finish an exercise, immediately ask an instructor to check your answer. Answers checked after the end of the class will have a 50% penalty.

If you are stuck or have any questions then raise your hand and an instructor will try to help you.

1 Mathematical functions [3 points]

The following code defines a function that calculates and returns the circumference of a circle, then tests it with the unit circle:

```
import math
def circle_circumference(radius):
    return 2 * math.pi * radius
print(circle_circumference(1))    # 6.283185307179586
```

Define the following functions (in one program) and test them by printing the result of calling them with the indicated arguments.

function	calculation	test values
<code>circle_area(r)</code>	πr^2	1 2 3
<code>sphere_volume(r)</code>	$\frac{4}{3}\pi r^3$	1 2 3
<code>triangle_area(a, b)</code>	$\frac{1}{2}ab$	1, 1 2, 2 3, 4
<code>hypotenuse(a, b)</code>	$\sqrt{a^2 + b^2}$	1, 1 3, 4 5, 12
<code>point_distance(x1, y1, x2, y2)</code>	$\sqrt{(x1 - x2)^2 + (y1 - y2)^2}$	0, 0, 1, 1 3, 4, 6, 8 8, 6, 4, 3

▷▷ Ask an instructor to check and record your work **now** (before you run out of time).

2 A string function [3 points]

Write a function called `justify(s)` that adds spaces to the *left* of the string `s` so that the total number of characters in the result is 10. For example:

```
justify("hello")           ⇒ "      hello"
len(justify("hello"))      ⇒ 10
```

Hints:

- The built-in function `len(s)` tells you how many characters are in the string `s`.
- If the length of the string `s` is n then adding $10 - n$ spaces to the beginning of `s` will make a new string exactly 10 characters long.
- Multiplying a string `s` by an integer n gives you a new string containing n copies of `s`. For example:
`3 * " " == " "`
- Adding two strings `s` and `t` together creates a new string containing `s` followed by `t`. For example:
`3 * "o" + "ps!" == "ooops!"`

Test your function as follows:

```
print(justify("one"))    ⇒ "      one"
print(justify("two"))   ⇒ "      two"
print(justify("three")) ⇒ "     three"
print(justify("four"))  ⇒ "      four"
print(justify("five"))  ⇒ "      five"
```

▷▷ Ask an instructor to check and record your work **now** (before you run out of time).

3 Hierarchical decomposition [3 points]

Write a function that returns a string containing the current time in 'alarm clock' format, e.g., "12:34:56".

Let's make this easy by using *hierarchical decomposition* to turn a big, difficult problem into several smaller, easy problems. The parts we need, from the bottom up, are:

- A function `pad(n)` that converts a number between 0 and 59 into a two-character time component by adding a '0' before `n` when `n` is a single digit.
- A function `alarm_format(h, m, s)` that calls `pad()` on `h`, `m`, and `s` to add leading zeros where needed, then returns the three results separated by two ':' characters.
- A function `time_string()` that finds the hour, minute, and second of the day and then uses `alarm_format()` to build an 'alarm format' representation of the current time.

3.1 Adding leading zeros to a number

Write a function called `pad(n)` that adds zeros "0" to the left of the number `n` so that the total number of digits in the result is 2.

Hints:

- You will have to convert `n` to a string before you can add any required leading zero.
- The built-in function `str(n)` returns a string representing the number `n`.
- The rest can be almost identical to `justify(s)`, with just two trivial changes.

Test your function with:

```
print(pad(""))          ⇒ "00"
print(pad("2"))         ⇒ "02"
print(pad(2))           ⇒ "02"
print(pad("42"))        ⇒ "42"
print(pad(42))          ⇒ "42"
```

3.2 Formatting hours, minutes, and seconds as a time string

Write a function `alarm_format(h, m, s)` that converts the numbers `h`, `m`, and `s` into a string representing a time in 'alarm clock' format. Test your function with:

```
print(alarm_format(13, 2, 7)) ⇒ "13:02:07"
```

3.3 Formatting the current time

Python has a module called `datetime` that provides functions and values related to time. You can obtain a value representing the current time and then ask it for the hour, minute, and second of the day like this:

```
import datetime
now = datetime.datetime.now()
print(now.hour, now.minute, now.second)  # 9 12 48
```

(Try running the last two lines a few times. You should see at least the seconds changing.)

Write a function `time_string()` that gets the current time from `datetime` (as shown above) and then uses `alarm_format()` to present the `hour`, `minute`, and `second` part of the current time in the format we desire. Return the formatted time string as the result of `time_string()`. Test your function with:

```
print(time_string()) ⇒ "09:19:48"
```

(Of course, your result should show your actual current time.)

▷▷ Ask an instructor to check and record your work now (before you run out of time).

4 Functions as ‘first class’ values [1 point]

Functions are values too. You can pass a function as an argument to another function, or return one as its result. Try this

```
print(print)
```

or even:

```
p = print
p(p)
```

Write a function `twice(f, x)` that calls `f(x)` twice. Test your function with:

```
twice(print, "hello") ⇒ "hello"
                        "hello"
```

▷▷ Ask an instructor to check and record your work now (before you run out of time).

5 Challenges [1 bonus point]

Complete all the challenges before the end of the class to earn a bonus point.

5.1 Default argument values

To specify a default value for an argument you can ‘assign’ the desired default value to its parameter in the parameter list, like this:

```
def functionName(arg = defaultValue): ...
```

For example:

```
def functionName(arg = 42):
    print("the argument is " + str(arg))

functionName("hello") # => the argument is hello
functionName()        # => the argument is 42
```

Write a function `double(n)` that doubles its argument. If you do not supply an argument then the function should assume you meant `1`.

5.2 A more general `pad()` function

Make your `pad(n, width, character)` function more general by specifying three parameters: a value `n` (integer or string) to be padded, the desired `width` of the result, and the `character` to use for padding. Test your function using these statements:

```
print(pad("hello", 10, ' ')) # "      hello"
print(pad("", 2, '0'))       # "00"
print(pad(4, 2, '0'))        # "04"
print(pad(42, 2, '0'))       # "42"
print(pad("", 8, "-"))        # "-----"
print(pad(123, 8, "."))      # "....123"
print(pad(456789, 8, "0"))   # "00456789"
```

5.3 Defaults for missing `pad()` arguments

Combine the previous two solutions to make `pad(n, width, character)` use a field `width` of `8` and a space `' '` as the `pad character`, by default, so that: `pad(123456) ⇒ " 123456"`

5.4 Generalising the `twice()` function

Given the function `twice(f, x)` that we wrote earlier, it would be nice if we could do something four times like this:

```
twice(twice, print, "cool!")
```

This does not work because `twice(f, x)` passes only one argument to the function `f`.

Python lets you ‘pack’ multiple arguments into a single parameter and then ‘unpack’ that parameter back into the original multiple arguments when (e.g.) calling another function. The syntax is:

```
def functionName(*args):
    print("args are:", *args)

functionName("hello", 42, "way", "cool!") # => args are: hello 42 way cool!
```

Note that you can put ‘normal’ parameter names before the final ‘packed’ parameter name, if you like.

Rewrite `twice(f, x)` so that instead of a single argument `x` it will pass any number of arguments to `f` both times it is called. Test your function with:

```
twice(twice, print, "cool!") ⇒ cool!  
                                cool!  
                                cool!  
                                cool!
```

Without trying it, can you predict the result of: `twice(twice, twice, print, "cooler!")` ?

▷▷ Show your challenge solutions to an instructor and claim your bonus point.